

Path-Coloring Algorithms for Plane Graphs

Aven Bross

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

dabross@alaska.edu

Glenn G. Chappell

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

chappellg@member.ams.org

Chris Hartman

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

cmhartman@alaska.edu

February 20, 2025

2010 Mathematics Subject Classification. Primary 05C38; Secondary 05C10, 05C15.

Key words and phrases. Path coloring, list coloring, algorithm.

Abstract

A path coloring of a graph G is a vertex coloring of G such that each color class induces a disjoint union of paths. We present two efficient algorithms to construct a path coloring of a plane graph.

The first algorithm, based on a proof of Poh, is given a plane graph, and produces a path coloring of the given graph using three colors.

The second algorithm, based on similar proofs by Hartman and Škrekovski, performs a list-coloring generalization of the above. The algorithm is given a plane graph and an assignment of lists of three colors to each vertex, and it produces a path coloring of the given graph in which each vertex receives a color from its list.

Implementations are available for all algorithms that are described.

1 Introduction

A (*finite, simple*) graph $G = (V, E)$ is a finite set $V(G)$ of *vertices* and a set $E(G)$ of *edges*: unordered pairs of vertices. We write $n(G) = |V(G)|$ and $m(G) = |E(G)|$ for the number of vertices and edges, respectively. Vertices $u, v \in V(G)$ are *neighbors* if $uv \in E(G)$; we

say u, v are *incident* to uv . We write $N(v)$ for the set of neighbors of v . The degree of v is $\deg(v) = |N(v)|$. We write $v \in G$ or $uv \in G$ if it is clear v is a vertex or uv is an edge.

We say H is a subgraph of G if $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$. If $S \subseteq V(G)$, the *induced subgraph* of S in G consists of S and all edges in $E(G)$ with both incident vertices in S . A subgraph is induced in G if it is the induced subgraph of its vertex set.

If H, K are subgraphs of G , the union $H \cup K$ is the union of their edge and vertex sets, and the difference $H - K$ is the induced subgraph of $V(H) \setminus V(K)$ in H . If $u, v \in V(H)$, $uv \in E(G)$, we define $H \cup uv$ and $H - uv$ by adding/removing the edge uv to/from $E(H)$.

A graph is *planar* if it may be drawn in the plane by mapping each vertex to a point and each edge to a curve connecting the points of its incident vertices such that curves intersect only at their endpoints. A *face* is a maximal region of the plane not containing any point of the drawing; we will always refer to a face by the vertices and edges mapped on its border. The unbounded face is the *outer face*. A (*combinatorial*) *embedding* (or *rotation scheme*) is an equivalence class of drawings determined by the circular ordering of edges around each vertex. A *plane graph* is a planar graph with a fixed embedding. Subgraphs of a plane graph are also plane graphs with the inherited embedding.

A *path* $P = v_1, v_2, \dots, v_k$ is the graph with $V(P) = \{v_1, v_2, \dots, v_k\}$ and $E(P) = \{v_1v_2, v_2v_3, \dots, v_{k-1}v_k\}$; we say P is a v_1, v_k -path. A *cycle* $C = v_1, v_2, \dots, v_k$ is the union of the v_1, v_k -path and the edge v_1v_k .

A (*vertex*) *coloring* of a graph G is a map $c : V(G) \rightarrow \mathbb{N}$; a k -coloring, $k \in \mathbb{N}$, is a map $c : V(G) \rightarrow \{1, 2, \dots, k\}$. The *color class* of $d \in \mathbb{N}$ is the set $c^{-1}(d) \subseteq V(G)$. A *path coloring* is a coloring where each color class induces a union of disjoint paths (more commonly a *disjoint union* of paths, but union and disjoint union coincide for disjoint subgraphs). A graph G is *path k -colorable* if G admits a path k -coloring. A *list assignment* of a graph G is a map $L : V(G) \rightarrow \mathcal{P}_{\text{fin}}(\mathbb{N})$ assigning a finite set of colors to each vertex. An *L -coloring* (or *L -list-coloring*) of G is a coloring $c : V(G) \rightarrow \mathbb{N}$ such that $c(v) \in L(v)$ for each $v \in V(G)$. A graph is *path k -choosable* if it admits a path L -coloring for any list assignment L with at least k colors in each list, i.e. $|L(v)| \geq k$ for each $v \in G$.

See West [12] for a more thorough overview of graph theoretic terms.

Broere & Mynhardt conjectured [2, Conj. 16] that every planar graph is path 3-colorable. This was proven independently by Poh [9, Thm. 2] and by Goddard [6, Thm. 1].

Theorem 1.1 (Poh 1990, Goddard 1991). *If G is a planar graph, then G is path 3-colorable.* $\square$

It is easily shown that the “3” in Theorem 1.1 is best possible. In particular, Chartrand & Kronk [5, Section 3] gave an example of a planar graph whose vertex set cannot be partitioned into two subsets, each inducing a forest.

Hartman [7, Thm. 4.1] proved a list-coloring generalization of Theorem 1.1 (see also Chappell & Hartman [4, Thm. 2.1]).

Theorem 1.2 (Hartman 1997). *If G is a planar graph, then G is path 3-choosable.* $\square$

Essentially the same technique was used by Škrekovski [11, Thm. 2.2b] to prove a result slightly weaker than Theorem 1.2.

We discuss two path coloring algorithms based on proofs of the above theorems.

In Section 2 we outline our graph representations and the basis for our computations of time complexity.

Section 3 covers an algorithm based on Poh's proof of Theorem 1.1. The algorithm is given a plane graph; it produces a path coloring of the given graph using three colors.

Section 4 covers an algorithm based Hartman's proof of Theorem 1.2, along with the proof of Škrekovski mentioned above. The algorithm is given a plane graph and an assignment of a list of three colors to each vertex; it produces a path coloring of the given graph in which each vertex receives a color from its list.

Section 5 provides benchmark results for implementations of each algorithm [3]. The section also discusses how each algorithm may be modified to benefit from parallelism.

2 Graph Representations and Time Complexity

A graph G is *connected* if given any $u, v \in G$, there exists a u, v -path in G , and *biconnected* if $G - v$ is connected for every $v \in G$. A plane graph G is *triangulated* if it is connected and $n(G) \leq 2$, or if every face is a *triangle* (a cycle with 3 vertices). If every face but (maybe) the outer face is a triangle, then G is *weakly triangulated*; a triangulated graph is weakly-triangulated. If G is biconnected and $n(G) \geq 3$, then its outer face is a cycle.

The input for each algorithm will be a biconnected, weakly triangulated plane graph G with n vertices and m edges. The input size will be n . By Euler's formula $m \leq 3n - 6$, with $m = 3n - 6$ if G is triangulated, so $\mathcal{O}(m) = \mathcal{O}(n)$ and it is equivalent to take the input size to be m . Arbitrary planar graphs may be plane embedded and triangulated (by adding edges) in $\mathcal{O}(n)$ time, see e.g. Hopcroft and Tarjan [8] or Boyer and Myrvold [1].

We will represent G for computation via *adjacency lists*: a list, for each vertex v , of the neighbors of v . A vertex is represented by an integer $0 \dots n - 1$. The order of each list represents the embedding. In algorithms $N(v)$ will refer to the adjacency list.

Let $v \in G$ and $u \in N(v)$. Define $\text{next}_v(u) \in N(v)$ to be the neighbor counter-clockwise from u around v , and $\text{prev}_v(u) \in N(v)$ the neighbor clockwise from u . In algorithms ${}^v u$ will be a reference to u 's entry in $N(v)$, i.e. ${}^v u$ represents both u and the position of u in v 's adjacency list; define $\text{next}({}^v u) = {}^v(\text{next}_v(u))$ and $\text{prev}({}^v u) = {}^v(\text{prev}_v(u))$.

In Section 3 we'll only need our representation to be able to compute $\text{next}({}^v u)$ in constant time from ${}^v u$. In Section 4 however, we'll also need the ability to find $\text{prev}({}^v u)$ and the *back reference* ${}^u v$ from ${}^v u$ in constant time. Representing $N(v)$ via an array or doubly linked list solves computing $\text{prev}({}^v u)$, but back references require a different representation. We define an *augmented adjacency list* to be an adjacency list that stores the pair $(u, {}^u v)$ for each neighbor u of v , rather than just the vertex u . Given an adjacency list representation, an augmented representation may be constructed in $\mathcal{O}(m)$ time.

Algorithm 2.1: Augment Adjacency Lists

Input. An adjacency list representation N of a graph G .

Output. An augmented adjacency list representation N^* of G with the same rotation scheme as N .

Step 1. For each $v \in G$ construct an augmented adjacency list $N^*(v)$ with the rotation scheme of $N(v)$, leaving the uv portion of each entry $(u, {}^uv)$ in $N^*(v)$ uninitialized.

Step 2. For each vertex v construct an array $\text{Wrk}(v)$ of vertex-reference pairs with length $\deg(v)$. For each v from 0 to $n - 1$ iterate through $N^*(v)$. For each neighbor u in $N^*(v)$ append the pair $(v, {}^vu)$ to $\text{Wrk}(u)$, where vu is a reference to u 's entry $N^*(v)$.

Step 3. For each v from $n - 1$ to 0 iterate through $\text{Wrk}(v)$. Upon reaching a pair $(u, {}^uv)$ in $\text{Wrk}(v)$ the last element of $\text{Wrk}(u)$ will be $(v, {}^vu)$; see the below. Lookup the entry vu in $N^*(v)$ and initialize the back reference uv , and vice versa. Remove $(v, {}^vu)$ from the back of $\text{Wrk}(u)$.

After completing Step 2 in Algorithm 2.1 the array $\text{Wrk}(v)$ will contain $(u, {}^uv)$ for each neighbor u of v , sorted in increasing order by the neighbor u . Suppose that v is the current vertex at a given iteration of Step 3 in Algorithm 2.1. For each edge $uw \in E(G)$ such that $u < w$ and $v < w$, prior iterations of Step 3 will have initialized the back references for wu in $N^*(u)$ and uw in $N^*(w)$, and removed the pair $(w, {}^wu)$ from $\text{Wrk}(u)$. Therefore for each $(u, {}^uv)$ in $\text{Wrk}(v)$, the array $\text{Wrk}(u)$ will contain only entries for vertices w where $w \leq v$. So the last element of $\text{Wrk}(u)$ must be $(v, {}^vu)$.

The explicit $(u, {}^uv) \in N^*(v)$ notation above was to distinguish between the two representations: going forward, if an algorithm takes as input a graph represented by augmented adjacency lists, we'll simply use $u \in N(v)$ to refer to a neighbor u in v 's list.

3 Path Coloring

In this section we describe a linear time algorithm to path 3-color plane graphs. Let's first recount Poh's path 3-coloring strategy.

Let C be a cycle in a plane graph G . If $u, v \in C$, then we define $C[u, v]$ to be the clockwise u, v -path around C ; if $u = v$, then $C[u, u] = u$. Define $\text{Int}(C)$ to be the subgraph of G consisting of C and all vertices and edges interior to C ; equivalently, $\text{Int}(C)$ is the maximal subgraph of G with outer face C .

Let P be a path in a plane graph G . If $u, v \in P$, then we define $P[u, v]$ to be the u, v -path in P . We define $\text{Int}(P) = P$.

Lemma 3.1 (Poh 1990). *Let G be a biconnected, weakly triangulated plane graph with outer face C . Let $c : V(C) \rightarrow S \subsetneq \{1, 2, 3\}$ be a 2-coloring of C such that each color class induces a nonempty path. There exists an extension of c to a path 3-coloring $c : V(G) \rightarrow \{1, 2, 3\}$ such that for each $v \in G - C$, if $vu \in E(G)$ with $u \in C$, then $c(v) \neq c(u)$.*

Proof. If $n(G) \leq 3$, then $G = C$ and the path 2-coloring of C is a path 3-coloring of G . We proceed by induction on the order of G .

Let P_1, P_2 be the two paths induced by the 2-coloring of the outer face C . We may write $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for vertices $u_1, u_2, w_1, w_2 \in C$.

Suppose C is an induced subgraph of G . Let $u, w \in G - C$ be the vertices such that the cycles u, u_2, u_1 and w, w_1, w_2 are faces of G ; note that u and w are uniquely determined, but it may be that $u = w$. Since C is an induced cycle in G and $G \neq C$, $G - C$ is connected.

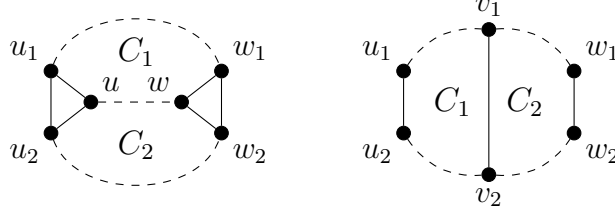


Figure 1: The proof of Lemma 3.1 when C is induced (left) or has a chord (right).

Let P_3 be a u, w -path in $G - C$ of minimal length, and note that P_3 is an induced subgraph of $G - C$. Color each vertex of P_3 with the color in $\{1, 2, 3\} \setminus S$ not used in the 2-coloring of C . Let $C_1 = P_1 \cup P_3 \cup uu_1 \cup ww_1$ and $C_2 = P_2 \cup P_3 \cup uu_2 \cup ww_2$. The subgraphs $\text{Int}(C_1)$ and $\text{Int}(C_2)$ together with the coloring c each satisfy the requirements of the lemma. By the inductive hypothesis there exist extensions of c to a path 3-coloring of $\text{Int}(C_1)$ and a path 3-coloring of $\text{Int}(C_2)$. Since $\text{Int}(C_1)$ and $\text{Int}(C_2)$ only share the vertices in P_3 on their respective outer faces, the colorings agree and form a path 3-coloring of G .

Suppose C is not an induced subgraph. Then there exists an edge $v_1v_2 \in E(G) \setminus E(C)$ with $v_1 \in P_1$ and $v_2 \in P_2$. Let $C_1 = C[v_2, v_1] \cup v_1v_2$ and $C_2 = C[v_1, v_2] \cup v_1v_2$. By the inductive hypothesis, there exists an extension of c to a path 3-coloring of $\text{Int}(C_1)$ and $\text{Int}(C_2)$. Again the subgraphs only share vertices on their outer faces, and thus the colorings agree and together form a path 3-coloring of G . $\square$

Let G be a triangulated plane graph. We may trivially path 2-color the outer triangle. By Lemma 3.1 this extends to a path 3-coloring of G . If G is an arbitrary planar graph, we may compute an embedding and add edges to produce a triangulated plane graph G' . Theorem 1.1 follows immediately as any path coloring of G' is also a path coloring of G .

A natural way to implement Poh's algorithm is to locate u , and then use a breadth-first search to construct a minimal u, w -path and/or locate a chord edge, see Algorithm 3.2.

Algorithm 3.2: Path 3-Color (BFS)

Input. Let C be a cycle or length 2 path in a weakly triangulated plane graph G represented via adjacency lists. Let c be a length n array representing a 2-coloring of C such that each color class induces a nonempty path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for $u_1, u_2, w_1, w_2 \in C$. Assume $c(v) = 0$ for each $v \in \text{Int}(C) - C$.

The concrete input will be G, c , and the tuple (u_1, w_1, u_1u_2, w_2) .

Output. For each vertex $v \in \text{Int}(C) - C$, a color in $\{1, 2, 3\}$ will be assigned to $c(v)$ such that c represents a path 3-coloring of $\text{Int}(C)$ extending the original 2-coloring of C . Moreover, if $v \in \text{Int}(C) - C$ has a neighbor $u \in C$, then $c(v) \neq c(u)$.

Base case. If $u_1 = w_1$ and $u_2 = w_2$, then C is a length 2-path and $\text{Int}(C) = C$.

Recursive case. Let $u_1u \leftarrow \text{next}(u_1u_2)$. Note that u, u_2, u_1 is a face of G , see Figure 2.

Case 1. Suppose that $c(u) \neq 0$. If $c(u) = c(u_2)$, then $u \in P_2$. Make a recursive call with the input (u_1, w_1, u_1u, w_2) to color $\text{Int}(C_1)$ where $C_1 = C[u_1, u] \cup u_1u$. Otherwise,

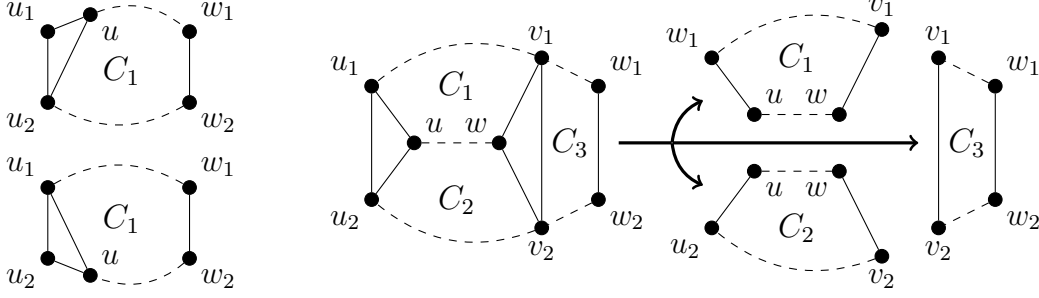


Figure 2: Algorithm 3.2, Case 1 (left) and Case 2 (right).

$c(u) = c(v_1)$ and $u \in P_1$. Iterate through $N(u)$ to find ${}^u u_2$. Make a recursive call with the input $(u, w_1, {}^u u_2, w_2)$ to color $\text{Int}(C_1)$, where $C_1 = C[u, u_2] \cup uu_2$.

Case 2. Suppose that $c(u) = 0$, i.e. $u \in \text{Int}(C) - C$. Perform a breadth-first search in $\text{Int}(C) - C$ starting from u ; such a search may be implemented by visiting $v \in G$ only if $c(v) = 0$. Terminate the search upon finding a vertex w with a pair of subsequent neighbors ${}^w v_2$ and ${}^w v_1 = \text{next}({}^w v_2)$ such that $c(v_2) = c(u_2)$ and $c(v_1) = c(u_1)$, i.e. $v_2 \in P_2$ and $v_1 \in P_1$. Backtrack along the search tree from w to find a minimal u, w -path P_3 . For each vertex $v \in P_3$, assign $c(v) \leftarrow d$, where $d \in \{1, 2, 3\} \setminus \{c(u_1), c(u_2)\}$.

Iterate through $N(u)$ and $N(v_1)$ to find ${}^u u_2$ and ${}^{v_1} v_2$. To color the remaining vertices we make three recursive calls:

1. with $(u_1, v_1, {}^{u_1} u, w)$ to color $\text{Int}(C_1)$, where $C_1 = C[u_1, v_1] \cup P_3 \cup v_1 u \cup v_1 w$;
2. with $(u, w, {}^u u_2, v_2)$ to color $\text{Int}(C_2)$, where $C_2 = C[v_2, u_2] \cup P_3 \cup uu_2 \cup ww_2$;
3. with $(v_1, w_1, {}^{v_1} v_2, w_2)$ to color $\text{Int}(C_3)$, where $C_3 = C[v_1, v_2] \cup v_1 v_2$.

All remaining vertices in $\text{Int}(C) - C - P_3$ are now colored and we are done.

Let G be a triangulated plane graph and let $C = v_1, v_2, v_3$ be the outer triangle of G labeled in clockwise order. We may compute a path 3-coloring c of G as follows. Create a length n integer array c and initialize each entry to zero. Assign $c(v_1) \leftarrow 1$, $c(v_2) \leftarrow 2$, and $c(v_3) \leftarrow 2$. Finally, apply Algorithm 3.2 with input G , c , and $(v_1, v_1, {}^{v_1} v_2, v_3)$.

Unfortunately, Algorithm 3.2 is not linear. Consider the sequence of “pyramid graphs” $\{A_k\}_{k \in \mathbb{N}}$ depicted in Figure 3. Fix $k \in \mathbb{N}$ and note that $n = n(A_k) = k(k+1)/2 + 3$. Assume that the outer triangle is 2-colored such that the top vertex is assigned a color distinct from the bottom two. At recursive depth i , Algorithm 3.2 will perform a breadth first search of the pyramid-shaped interior subgraph with sides of length $r = k - i$, starting from the vertex in the bottom left corner. The search will hit all $r(r+1)/2$ vertices of the interior pyramid. Therefore the total number of operations performed will be

$$\Omega \left(\sum_{r=1}^k \frac{r(r+1)}{2} \right) = \Omega(k^3) = \Omega(n^{3/2}).$$

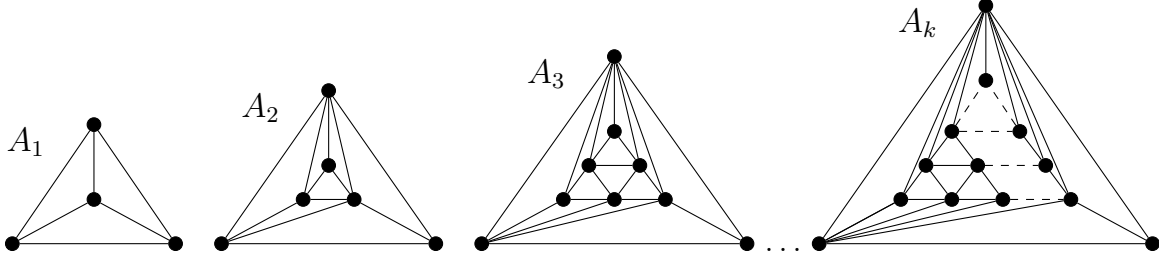


Figure 3: The sequence of “pyramid graphs” on which Algorithm 3.2 performs poorly.

The correctness of Poh’s proof, however, relied only on locating an induced u, w -path. Algorithm 3.3 describes a linear time implementation of Poh’s algorithm that locates and colors an induced u, w -path that is not always of minimal length.

For a subgraph H of G , define $N(H)$ to be the vertices in $G - H$ with a neighbor in H . For $P_1 = C[u_1, w_1]$ let $p_1 : N(P_1) \rightarrow V(P_1)$ be the function that maps each vertex to its furthest clockwise neighbor along P_1 , i.e. $V(C[p_1(v), w_1]) \cap N(v) = \{p_1(v)\}$.

Algorithm 3.3: Path 3-Color (Neighbor Trace)

Input. Let C be an induced cycle in a weakly triangulated plane graph G represented via adjacency lists. Let c be a length n array representing a 2-coloring of C such that each color class induces a nonempty path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$, where $u_1, u_2, w_1, w_2 \in C$. Assume that $c(v) = 0$ for each $v \in \text{Int}(C) - C$. Let S be a length n integer array and let ℓ_1 be an integer such that for each $v \in \text{Int}(C) - C$ we have $S(v) = \ell_1$ if and only if $v \in N(P_1)$.

The concrete input will be G, c and S , along with the ordered pair $(u_1, {}^{u_1}u_2, \ell_1)$.

Output. For each vertex $v \in \text{Int}(C) - C$ a color in $\{1, 2, 3\}$ will be assigned to $c(v)$ such that c represents a path 3-coloring of $\text{Int}(C)$ extending the original 2-coloring of C . Moreover, if $v \in \text{Int}(C) - C$ has a neighbor $u \in C$, then $c(v) \neq c(u)$.

Procedure. Assign ${}^{u_1}u \leftarrow \text{next}({}^{u_1}u_2)$.

Base case. If $c(u) \neq 0$, then $u = w_2$ or $u = w_1$. Thus C is a face and we are done.

Recursive case. Suppose that $c(u) = 0$, i.e. $u \in \text{Int}(C) - C$. Let $d \in \{1, 2, 3\} \setminus \{c(u_1), c(u_2)\}$. Let $\ell_3 = \max_{v \in G} S(v) + 1$. We will construct an induced u, w -path $P_3 = v_1, v_2, \dots, v_j$ such that $v_1, v_2, \dots, v_j \in N(P_1)$, and color each $v \in P_3$ with $c(v) \leftarrow d$. We’ll also mark each $v \in N(P_3) \setminus V(C)$ with $S(v) \leftarrow \ell_3$, and make recursive calls to color $\text{Int}(C) - C - P_3$. It may be useful to refer to Figures 4 and 5.

Step 1. Assign $j \leftarrow 1$ and $v_j \leftarrow u$; here j, v_j store the end of the path P_3 that we’ve constructed so far. Set $v_0 \leftarrow u_2$ so that v_{j-1} is defined for $j = 1$. Assign $i \leftarrow 1$ and $v_i s \leftarrow {}^{v_i}u_2$; here $i, {}^{v_i}s$ store the last edge $v_i s$ that was visited where $v_i \in P_3$ and $s \in P_2$.

Loop 2. We’ll build the path P_3 , adding one vertex per iteration. Assign $c(v_j) \leftarrow d$. Find ${}^{v_j}v_{j-1}$ in $N(v_j)$ and set ${}^{v_j}v \leftarrow \text{next}({}^{v_j}v_{j-1})$. Set $y \leftarrow \text{NULL}$; here y will store the first neighbor of v_j counter-clockwise from v_{j-1} that is in $N(P_1) \setminus V(C)$ (if any). Set ${}^{v_j}t \leftarrow \text{NULL}$; here ${}^{v_j}t$ will store the last edge $v_j t$ encountered such that $t \in P_1$ (if any).

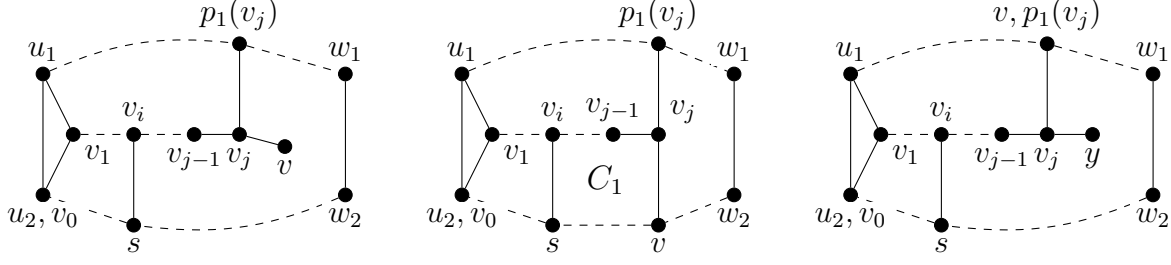


Figure 4: Algorithm 3.3, Case 2.1.1 (left), Case 2.1.2 (middle), and Case 2.1.3 (right).

Loop 2.1. We'll first consider the neighbors of v_j between v_{j-1} and $p_1(v_j)$ counter-clockwise. Each iteration will handle the current neighbor ${}^{v_j}v$. We'll make a recursive call for each induced cycle C_1 we discover with $V(C_1) \subseteq V(P_2) \cup V(P_3)$. Refer to Figure 4.

Case 2.1.1. Suppose that $c(v) = 0$, i.e. $v \in \text{Int}(C) - C$. If $S(v) = \ell_1$, i.e. $v \in N(P_1)$, and $y = \text{NULL}$, then set $y \leftarrow v$. Set $S(v) \leftarrow \ell_3$ and ${}^{v_j}v \leftarrow \text{next}({}^{v_j}v)$. Go back to Loop 2.1.

Case 2.1.2. Suppose that $c(v) = c(u_2)$, i.e. $v \in P_2$. Make a recursive call with input $(v_i, {}^{v_i}s, \ell_3)$ to color $\text{Int}(C_1)$, where $C_1 = P_3[v_i, v_j] \cup C[v, s] \cup v_i s \cup v_j v$; see Figure 4 (middle). Assign $i \leftarrow j$, ${}^{v_i}s \leftarrow {}^{v_j}v$, and ${}^{v_j}v \leftarrow \text{next}({}^{v_j}v)$. Go back to Loop 2.1.

Case 2.1.3. Suppose that $c(v) = c(u_1)$, i.e. $v = p_1(v_j) \in P_1$. Assign ${}^{v_j}t \leftarrow {}^{v_j}v$ and ${}^{v_j}v \leftarrow \text{next}({}^{v_j}v)$. Proceed to Loop 2.2.

Note that if $y = \text{NULL}$, then $v_j = w$. To see this, observe that since $v \in P_1$, $\text{prev}_{v_j}(v) \in N(P_1)$, and $\text{prev}_{v_j}(v) \notin P_1$, we must have $\text{prev}_{v_j}(v) \in C - P_1 = P_2$.

Loop 2.2. We'll now consider the neighbors of v_j between $p_1(v_j)$ and v_{j-1} counter-clockwise. Each iteration will handle the current neighbor ${}^{v_j}v$. We'll make a recursive call for each induced cycle C_1 we discover with $V(C_1) \subseteq V(P_1) \cup V(P_3)$. Observe ${}^{v_j}t \neq \text{NULL}$.

Case 2.2.1. Suppose that $c(v) = 0$, i.e. $v \in \text{Int}(C) - C - P_3$. Assign $S(v) \leftarrow \ell_3$ and ${}^{v_j}v \leftarrow \text{next}({}^{v_j}v)$. Go back to Loop 2.2.

Case 2.2.2. Suppose that $c(v) \neq 0$, i.e. $v \in C$ or $v \in P_3$. Make a recursive call with input $(v_j, {}^{v_j}t, \ell_3)$ to color $\text{Int}(C_1)$, where both the definition of C_1 and the next steps to take depend on $c(v)$. Refer to Figure 5.

If $c(v) = c(u_1)$, i.e. $v \in P_1$, then $C_1 = C[v, t] \cup v_j \cup v_j t \cup v_j v$. Assign ${}^{v_j}t \leftarrow {}^{v_j}v$ and ${}^{v_j}v \leftarrow \text{next}({}^{v_j}v)$. Go back to Loop 2.2.

Otherwise, $v = v_{j-1}$ (either $j > 1$ and $v \in P_3$, or $j = 1$ so $v = u_2 \in P_2$ and $t = u_1$). Then $C_1 = C[p_1(v), t] \cup P_3[v, v_j] \cup v p_1(v) \cup v_j t$. Proceed to Step 2.3.

Step 2.3. If $y \neq \text{NULL}$, then assign $j \leftarrow j + 1$, $v_j \leftarrow y$, and go back to Loop 2. Otherwise, $v_j = w$ and we are done.

We claim that the path $P_3 = v_1, v_2, \dots, v_j$ in Algorithm 3.3 is always an induced path in G . Suppose that $v_k v_j \in E(G)$ with $0 < k < j - 1$. Observe that

$$D = C[u_2, p_1(v_{j-1})] \cup P_3[v_1, v_{j-1}] \cup u_2 v_1 \cup v_{j-1} p_1(v_{j-1})$$

is a cycle in G with $v_{k-1}, v_k, v_{k+1} \in D$ and $v_j \notin \text{Int}(D)$, see Figure 5 (right). Therefore

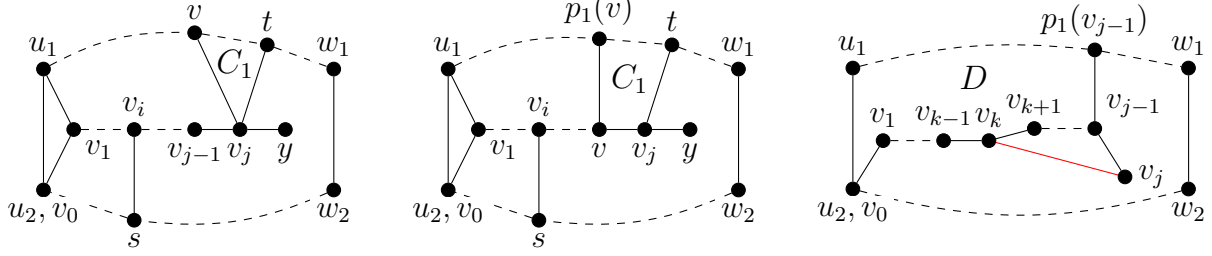


Figure 5: Algorithm 3.3, Case 2.2.2 when $v \in P_1$ (left) and when $v = v_{j-1}$ (middle). Also, a sketch of why P_3 is an induced path (right).

$v_k v_j \notin \text{Int}(D)$ and $v_k v_j$ lies between v_{k-1} and v_{k+1} counter-clockwise in $N(v_k)$. But v_{k+1} is the first neighbor in $N(P_1)$ counter-clockwise from v_{k-1} around v_k , a contradiction.

Given a triangulated plane graph G represented via adjacency lists, we can apply Algorithm 3.3 as follows. First, create length n integer arrays for c and S , initializing each entry to zero. Let $C = v_1, v_2, v_3$ be the outer triangle of G , labeled in clockwise order. Color $c(v_1) \leftarrow 1$, $c(v_2) \leftarrow 2$, and $c(v_3) \leftarrow 2$. Iterate through $N(v_1)$ and mark $S(v) \leftarrow 1$ for each neighbor v of v_1 . Then $G, c, S, (v_1, v_1 v_3, 1)$ form a valid input for Algorithm 3.3.

During Algorithm 3.3 we iterate through $N(v)$ for each $v \in G$ exactly twice: once to locate $v_j v_{j-1}$ when $v_j = v$ is colored and added to a path P_3 in Loop 2, and once to iterate through $N(v_j)$ in Loops 2.1 and 2.2. Therefore the time complexity is $\mathcal{O}(m) = \mathcal{O}(n)$.

4 Path List-Coloring

In this section we describe a linear time algorithm to path color plane graphs such that each vertex receives a color from a specified list. Hartman showed that this is always possible when each vertex is given a list of 3 colors [7, Thm. 4.1]. Around the same time Škrekovski proved a weaker result using the same coloring strategy [11, Thm. 2.2b].

The path list-coloring procedure discussed in this section is based on the constructive proofs found in Hartman and Škrekovski's papers. For those familiar with Hartman's proof: if $x = z$, then our G, x, y match those in [7, Lemma. 4.2], but our G is always biconnected; we follow Hartman's iterative procedure to color a path P clockwise from x along C , but we proceed inductively, one edge at a time, and simultaneously remove P from the subgraph in consideration. If $x \neq z$, then z is the furthest clockwise end of the path P being colored; the rest of P (including the original " x ") has been removed. The only manner in which our algorithm differs from Hartman's proof is that if x ever immediately follows y clockwise around C , then we swap x and y (see Case 1 below).

Given a graph G and a coloring c of G , for each $v \in G$ we define $\deg_c(v)$ to be the number of neighbors of v that share a color with v . Equivalently, $\deg_c(v)$ is the degree of v in the subgraph of G induced by the color class of $c(v)$.

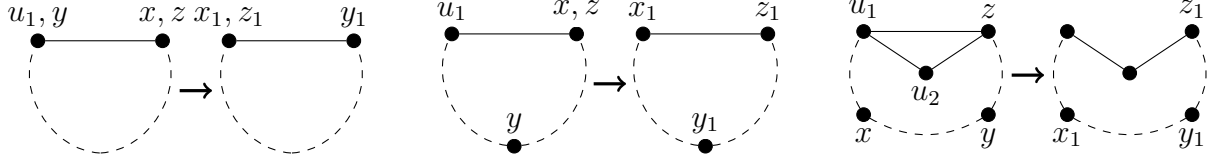


Figure 6: The proof of Lemma 4.1, Case 1 (left), Case 2 (middle), and Case 3.1 (right).

Lemma 4.1. *Let G be a biconnected, weakly triangulated plane graph with outer face C . Let $x, y, z \in C$ be vertices (not necessarily distinct) such that $z \in C[x, y]$. Let L be a list assignment of G such that*

$$\begin{aligned} |L(v)| &= 1 \text{ for } v \in \{x, y, z\}, \\ |L(v)| &\geq 2 \text{ for } v \in C - \{x, y, z\}, \\ |L(v)| &\geq 3 \text{ for } v \in G - C, \end{aligned}$$

and if $v \in C[x, z] - z$, then $L(v) \cap L(z) = \emptyset$. There exists a path L -coloring of G such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. Moreover, if $z = y$, or if $C[z, y] = z, y$ and $L(z) \cap L(y) = \emptyset$, then $\deg_c(z) = 0$.

Proof. Suppose that $m = |E(G)| \leq 1$. Then $G = C$. We define a path L -coloring c of G as follows. Let $c(x) \in L(x)$, $c(y) \in L(y)$, and $c(z) \in L(z)$ be the colors in each respective list. If $m = 0$, then $n(G) = 1$ and c is trivially a path L -coloring. Otherwise $m = 1$ and $n(G) = 2$. If $x \neq y$, then c is a path L -coloring of G with the constraints of the lemma. Otherwise the remaining $v \in G - y$ satisfies $|L(v)| \geq 2$ and we define $c(v) \in L(v) \setminus \{c(y)\}$.

We proceed by induction on m , the number of edges. Let $u_1, u_2, \dots, u_k$ label the neighbors of z in counter-clockwise order such that u_1 is the vertex immediately counter-clockwise from z around C and u_k is the vertex clockwise from z around C . In each of the subsequent cases we will apply the lemma separately to one or two subgraphs G_1, G_2 of G with corresponding labeled outer face vertices $x_i, y_i, z_i \in G_i$ for $i = 1, 2$.

Case 1. Suppose that $u_1 = y$. Note that $z = x$ since $z \in C[x, y]$. Therefore $V(C[z, y]) = V(C)$, so $|V(C[z, y])| \geq 3$ and $C[z, y] \neq z, y$. Apply the lemma to find a path L -coloring c of $G_1 = G$ with $x_1 = y$, $y_1 = x$, and $z_1 = y$.

Case 2. Suppose that $u_1 \neq y$ and $z = x$. Then $u_1 \in C - C[x, z]$ and $|L(u_1)| \geq 2$. Pick $d \in L(u_1) \setminus L(z)$. Define $L'(u_1) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_1$. Apply the lemma to find a path L' -coloring of $G_1 = G$ with $x_1 = u_1$, $y_1 = y$, and $z_1 = z$.

Case 3. Suppose that $u_1 \neq y$ and $z \neq x$. Our strategy will be to apply the inductive hypothesis to the *blocks* (maximal biconnected subgraphs) of $G - zu_1$. Because $u_1 \in C[x, z]$, it is guaranteed that $L(u_1) \cap L(z) = \emptyset$. Thus any path L -coloring of $G - zu_1$ is also a path L -coloring of G .

Case 3.1. Suppose that $u_2 \in G - C$. Define $L'(u_2) = L(u_2) \setminus L(z)$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Note that $|L'(u_2)| \geq |L(u_2)| - 1 \geq 2$. Let C_1 be the outer face of $G_1 = G - zu_1$, and observe that $C_1[x, z]$ is equal to $C[x, z]$ with the edge u_1z removed

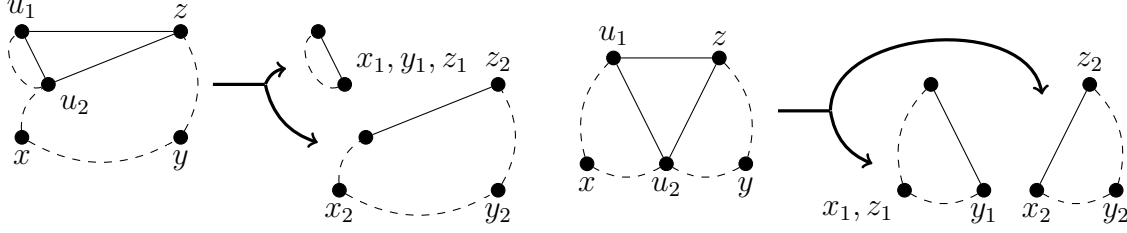


Figure 7: The proof of Lemma 4.1, Case 3.2.1 (left) and Case 3.2.2 (right).

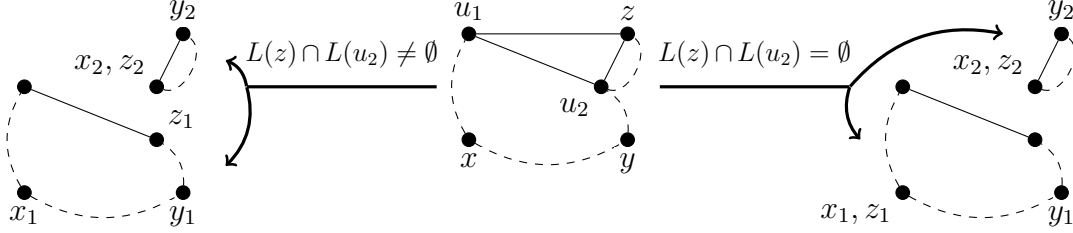


Figure 8: The proof of Lemma 4.1, Case 3.2.3.

and the path u_1, u_2, z added. Since $|L'(u_2)| \geq 2$ and $L'(u_2) \cap L'(z) = \emptyset$, we may apply the inductive hypothesis to find a path L' -coloring of G_1 with $x_1 = x, y_1 = y, z_1 = z$.

Case 3.2. Suppose that $u_2 \in C$. Observe that u_2 is a *cut-vertex* of $G - zu_1$, i.e. $G - zu_1 - u_2$ is disconnected. Define $C_1 = C[u_2, u_1] \cup u_1u_2$ and $C_2 = C[z, u_2] \cup zu_2$. Let $G_1 = \text{Int}(C_1)$ and $G_2 = \text{Int}(C_2)$. The subgraphs G_1 and G_2 are the two blocks of $G - zu_1$. Note that if $k = 2$, then $G_1 = G - z$ and $G_2 = z, u_2$.

In each subsequent case we will apply the inductive hypothesis to find a path L -coloring c_1 of G_1 and a path L -coloring c_2 of G_2 such that $c_1(u_2) = c_2(u_2)$. Let c be the L -coloring of G defined by $v \mapsto c_1(v)$ for $v \in G_1$ and $v \mapsto c_2(v)$ for $v \in G_2$. Observe that $\deg_c(v) = \deg_{c_1}(v)$ for each $v \in G_1 - u_2$, $\deg_c(v) = \deg_{c_2}(v)$ for each $v \in G_2 - u_2$, and $\deg_c(u_2) = \deg_{c_1}(u_2) + \deg_{c_2}(u_2)$. To show that c is a path L -coloring of G it suffices to show that $\deg_c(u_2) \leq 2$.

Case 3.2.1. Suppose that $u_2 \in C[x, z]$. Observe that $x, y, z \in C_2$. Apply the inductive hypothesis to find a path L -coloring c_2 of G_2 with $x_2 = x, y_2 = y$, and $z_2 = z$. Define $L'(u_2) = \{c_2(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_1 - u_2$. Find a path L' -coloring c_1 of G_1 with $x_1 = y_1 = z_1 = u_2$. Since $\deg_{c_1}(u_2) = 0$, it follows that c is a path L -coloring of G satisfying the constraints of the lemma.

Case 3.2.2. Suppose that $u_2 \in C[y, x] - x - y$. Pick $d \in L(u_2) \setminus L(z)$. Define $L'(u_2) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Observe that $x \in C_1$ and $z, y \in C_2$. Apply the inductive hypothesis to find a path L' -coloring c_1 of G_1 with $x_1 = z_1 = x$ and $y_1 = u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2, y_2 = y$, and $z_2 = z$. Note that $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) \leq 1$, thus c is a path L -coloring of G satisfying the constraints of the lemma.

Case 3.2.3. Suppose that $u_2 \in C[z, y]$. Note that $z \neq y$. There are two cases to

consider.

Case 3.2.3.1. Suppose that $L(z) \cap L(u_2) \neq \emptyset$. Then either $u_2 = y$ and $L(z) = L(y)$, or $C[z, y] \neq z, y$. Define $L'(u_2) = L(z)$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Find a path L' -coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Since $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) = 1$, it follows that c is a path L -coloring of G . If $u_2 = y$, then $z_1 = y_1 = y$ and $\deg_{c_1}(y) = 0$. So $\deg_c(y) \leq 1$ and c satisfies the constraints of the lemma.

Case 3.2.3.2. Suppose that $L(z) \cap L(u_2) = \emptyset$. Find a path L -coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = x$. Define $L'(u_2) = \{c_1(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_2 - u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Because $C[z_2, y_2] = z_2, y_2$ and $L'(z_2) \cap L'(y_2) = \emptyset$, we must have $\deg_{c_2}(u_2) = \deg_{c_2}(z_2) = 0$. Thus c is a path L -coloring of G .

Suppose $C[z, y] = z, y$. Then it must be that $u_2 = y$ and $k = 2$. Therefore $G_2 = z, y$ and $\deg_c(z) = 0$, since $c(z) \notin L(u_2)$. Thus c satisfies the constraints of the lemma. $\square$

Let G be a planar graph and L a list assignment such that $|L(v)| \geq 3$ for each $v \in G$. Compute a planar embedding and add edges to produce a triangulated plane graph G' . Pick $u \in G'$ on the outer face and $d \in L(u)$. Define the list assignment L' by $L'(u) = \{d\}$ and $L'(v) = L(v)$ for $v \in G' - u$. By Lemma 4.1 there exists a path L' -coloring c of G' , and thus a path L -coloring of G . So Theorem 1.2 follows immediately from Lemma 4.1.

The proof of Lemma 4.1 may be implemented as a linear time algorithm for plane graphs. For $v \in G$ and $u \in N(v)$, we will assume that $\text{next}^v(u)$, $\text{prev}^v(u)$, and ${}^v u$ can be computed in constant time from the entry ${}^v u$; see augmented adjacency lists in Section 2.

Algorithm 4.2: Path 3-Choose

Input. Let C be a cycle or length 2 path in a weakly triangulated plane graph G represented via augmented adjacency lists. Let $x, y, z \in C$ be such that $z \in C[x, y]$.

Let L be a length n array of lists of colors such that for $v \in \{x, y, z\}$ the list $L(v)$ has length one, for $v \in C - \{x, y, z\}$ the list $L(v)$ has length two or three, and for $v \in \text{Int}(C) - C$ the list $L(v)$ has length three. Furthermore, for each $v \in C[x, z] - z$ assume that $L(v)$ does not contain the color in $L(z)$.

Let B, F be length n arrays such that for each $v \in C$ we have $B(v) = {}^v b$ and $F(v) = {}^v f$, where b is the vertex immediately counter clockwise from v around C , and f is the vertex immediately clockwise from v around C .

Let S be a length n integer array such that, for $v \in \text{Int}(C)$, $S(v) \neq 0$ if and only if $v \in C$, and $S(v) = S(x)$ if and only if $v \in C[x, z] - z$ (or $v = x = z$). Let $\max(S) = \max_{v \in G} S(v)$ and $m_C = m(\text{Int}(C))$. If $z \neq x$, then define $\ell = \max(S)$. If $z = x$ and $C[y, x] \neq y, x$, then define $\ell = \max(S) + 1$. Otherwise, define $\ell = \max(S) + 2$. Assume that $\ell < 2(m - m_C) + 4$.

Let M be a length $2m + 2$ integer array such that if $v \in C$, then $M(S(v)) = S(y)$ if and only if $v \in C[z, y]$.

The concrete input will be G, L, B, F, S, M , and the tuple (x, y, z) .

Output. For each $v \in \text{Int}(C)$, all but one color will be removed from the list $L(v)$. The remaining color in each list will represent a path L -coloring of $\text{Int}(C)$ such that

$\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. If $z = y$, or if $C[z, y] = z, y$ and $L(z)$ did not contain the color in $L(y)$, then $\deg_c(z) = 0$. Moreover, afterwards $\max(S) \leq \ell + 2(m_C - 1)$.

Procedure. Assign ${}^z u_1 \leftarrow B(z)$. Our strategy will follow the proof of Lemma 4.1.

Base case. If $B(z) = F(z)$, then $C = z, u_1$ is a length 2 path. If $u_1 \neq x$ and $u_1 \neq y$, remove the color in $L(z)$ from $L(u_1)$, if it exists, then remove further colors until one color remains. Note that $m_C = 1$ and $\max(S) \leq \ell + 2(m_C - 1) = \ell$ by definition.

Recursive case. Suppose that $B(z) \neq F(z)$. Note that $n(C) > 2$ and therefore C is a cycle. Assign ${}^z u_2 \leftarrow \text{next}({}^z u_1)$. It will be immediately apparent that $\max(S) \leq \ell + 2(m_C - 1)$ after Cases 1, 2, and 3.1, so we will only argue that this bound is satisfied after Case 3.2. It may be useful to refer to Figures 6, 7, and 8.

Case 1. Suppose that $u_1 = y$. Then $z = x$, since $z \in C[x, y]$. Set $S(x) \leftarrow \max(S) + 1$ and $S(y) \leftarrow S(x)$. Assign $M(S(x)) \leftarrow S(x)$. Make a recursive call with the input (y, x, y) .

Case 2. Suppose that $z = x$ and $u_1 \neq y$. Remove the color in $L(z)$ from $L(u_1)$, if it exists, then remove further colors until one color remains. Assign $S(u_1) \leftarrow \max(S) + 1$ and $M(S(u_1)) \leftarrow S(u_1)$, and make a recursive call with input (u_1, y, z) .

Case 3. Suppose that $z \neq x$ and $u_1 \neq y$. Remove zu_1 by assigning $F(u_1) \leftarrow \text{prev}({}^{u_1} z)$ and $B(z) \leftarrow \text{next}({}^z u_1)$; refer to Figure 6 (right), Figure 7, and Figure 8.

Case 3.1. Suppose that $S(u_2) = 0$, that is, $u_2 \in \text{Int}(C) - C$. Assign $S(u_2) \leftarrow S(x)$ and initialize $B(u_2) \leftarrow {}^{u_2} u_1 = \text{next}({}^{u_2} z)$ and $F(u_2) \leftarrow {}^{u_2} z$; refer to Figure 6 (right). Remove the color in $L(z)$ from $L(u_2)$. Make a recursive call with the same input (x, y, z) .

Case 3.2. Suppose that $S(u_2) \neq 0$, that is, $u_2 \in C$. Just as in Case 3.2 of the proof of Lemma 4.1, we will separately consider the two blocks G_1 and G_2 of $\text{Int}(C) - zu_1$. Note that $m_C - 1 = m(G_1) + m(G_2)$.

As we've already removed the edge zu_1 by adjusting $B(z)$ and $F(u_1)$, it remains to "split" the neighborhood of u_2 at the edge ${}^{u_2} z$ such that the recursive call on G_1 considers only the neighbors of u_2 counter-clockwise from z , and the call on G_2 considers z and all neighbors clockwise from z ; refer to Figure 7 and Figure 8.

Store ${}^{u_2} b \leftarrow B(u_2)$ and ${}^{u_2} f \leftarrow F(u_2)$. To represent G_1 , assign $B(u_2) \leftarrow \text{next}({}^{u_2} z)$ and $F(u_2) \leftarrow {}^{u_2} f$. To represent G_2 , assign $B(u_2) \leftarrow {}^{u_2} b$ and $F(u_2) \leftarrow {}^{u_2} z$.

Case 3.2.1. Suppose that $S(u_2) = S(x)$, i.e. $u_2 \in C[x, z]$. Make a recursive call on G_2 with input (x, y, z) ; afterwards $\max(S) \leq \ell + 2m(G_1) - 2$. Assign $S(u_2) \leftarrow \max(S) + 1$ and $M(S(u_2)) \leftarrow S(u_2)$, and make a recursive call on G_1 with input (u_2, u_2, u_2) . Finally, observe that $\max(S) \leq \ell + 2m(G_1) - 2 + 1 + 2m(G_2) - 1 < \ell + 2(m_C - 1)$.

Case 3.2.2. Suppose that $z \neq x$ and $S(u_2) \neq 0$, but $S(u_2) \neq S(x)$ and $M(S(u_2)) \neq S(y)$. In other words, suppose that $u_2 \in C[y, x] - x - y$. First remove the color in $L(z)$ from $L(u_2)$, if it exists, then remove further colors until one remains. Set $S(u_2) \leftarrow S(x)$. Make a recursive call on G_1 with input (x, u_2, x) ; afterwards $\max(S) \leq \ell + 2m(G_1)$. Set $S(u_2) \leftarrow \max(S) + 1$ and $M(S(u_2)) \leftarrow S(u_2)$, and make a recursive call on G_2 with input (u_2, y, z) . Finally, observe that $\max(S) \leq \ell + 2m(G_1) + 1 + 2m(G_2) - 2 < \ell + 2(m_C - 1)$.

Case 3.2.3. Suppose that $z \neq x$ and $M(S(u_2)) = S(y)$, i.e. $u_2 \in C[z, y]$. There are two cases to consider.

Case 3.2.3.1. Suppose that the color in $L(z)$ is in $L(u_2)$. Remove all colors from $L(u_2)$ except for the color in $L(z)$. Make a recursive call on G_1 with input (x, y, u_2) ;

afterwards $\max(S) \leq \ell + 2m(G_1) - 2$. Assign $S(z) \leftarrow \max(S) + 1$, $S(u_2) \leftarrow S(z)$, and $M(S(z)) \leftarrow S(z)$. Make a recursive call on G_2 with input (u_2, z, u_2) . Finally, observe that $\max(S) \leq \ell + 2m(G_1) - 2 + 1 + 2m(G_2) - 1 < \ell + 2(m_C - 1)$.

Case 3.2.3.2. Suppose that the color in $L(z)$ is not in $L(u_2)$. Assign $M(S(x)) \leftarrow S(y)$. Make a recursive call on G_1 with input (x, y, x) ; afterwards $\max(S) \leq \ell + 2m(G_1)$. Assign $S(z) \leftarrow \max(S) + 1$, $S(u_2) \leftarrow S(z)$, and $M(S(z)) \leftarrow S(z)$. Make a recursive call on G_2 with input (u_2, z, u_2) . Observe $\max(S) \leq \ell + 2m(G_1) + 1 + 2m(G_2) - 1 = \ell + 2(m_C - 1)$.

Note that this is the only case where $M(S(v))$ is set a value other than $S(v)$. Setting $M(S(x)) \leftarrow S(y)$ effectively “joins” $C[x, u_1]$ onto $C[u_2, y]$ to represent $C_1[z_1, y_1]$ in G_1 , without having to re-mark each vertex in $C[x, u_1]$ with $S(y)$; see Figure 8.

Let G be a triangulated plane graph represented by augmented adjacency lists. Let L be a length n array of color lists of length 3, representing a list assignment of G . We may path L -color G using Algorithm 4.2 as follows.

Create length n arrays B, F and initialize $B(v)$ and $F(v)$ for each $v \in C$ on the outer face of G . Create a length n integer array S and initialize $S(v) \leftarrow 1$ for each $v \in C$ and $S(v) \leftarrow 0$ for each $v \in G - C$. Pick any $u \in C$, assign $S(u) \leftarrow 2$, and remove all but one color from $L(u)$. Create a length $2m + 2$ array of integers M and initialize $M(i) \leftarrow i$ for each $i \in \{0, 1, 2\}$. Finally, apply Algorithm 4.2 with the input G, L, B, F, S, M , and (u, u, u) . The remaining color in each list in L will represent a path L -coloring of G .

Every recursive case in Algorithm 4.2 performs a fixed number of constant time operations. Each case removes one edge from the graph representation, except for Case 1 and Case 2. In Case 1 a single recursive call is made with an input that will satisfy Case 2. In Case 2 a recursive call is made with an input that will not itself satisfy Case 1 or Case 2. Therefore the number of operations performed is $\mathcal{O}(m) = \mathcal{O}(n)$. See Figures 9, 10, and 11 for a concrete example of Algorithm 4.2.

5 Experimental Performance and Parallelism

A C implementation of each algorithm is available [3]. For each i from $3 \dots 7$, a uniform random sample of 100 plane triangulations of order 10^i was generated. Each algorithm was run on a each set of samples, see the timings in Figure 12 and the plot in Figure 14.

Plane triangulations were sampled using the method of Poulalhon and Schaeffer [10]. The vertices were randomly re-labeled in each sampled graph, and the corresponding adjacency list arrays were re-ordered in memory such that the list for newly-labeled vertex 0 came first in memory, followed by the list for vertex 1, and so on. The benchmark results therefore represent each algorithm’s performance on a set of uniformly sampled plane triangulations, each with a uniformly sampled representation layout in memory. The list assignments for Algorithm 4.2 were uniformly drawn from a set of six colors.

A breadth-first search (BFS) implemented with a circular array FIFO queue was run on the same set of graphs as a baseline algorithm that hits every half-edge of a graph exactly once. All binaries were compiled with Clang 19.1.7 and run on a Linux 6.12.9 machine with an x86_64 Intel N100 processor. Both source code and binaries are available [3].

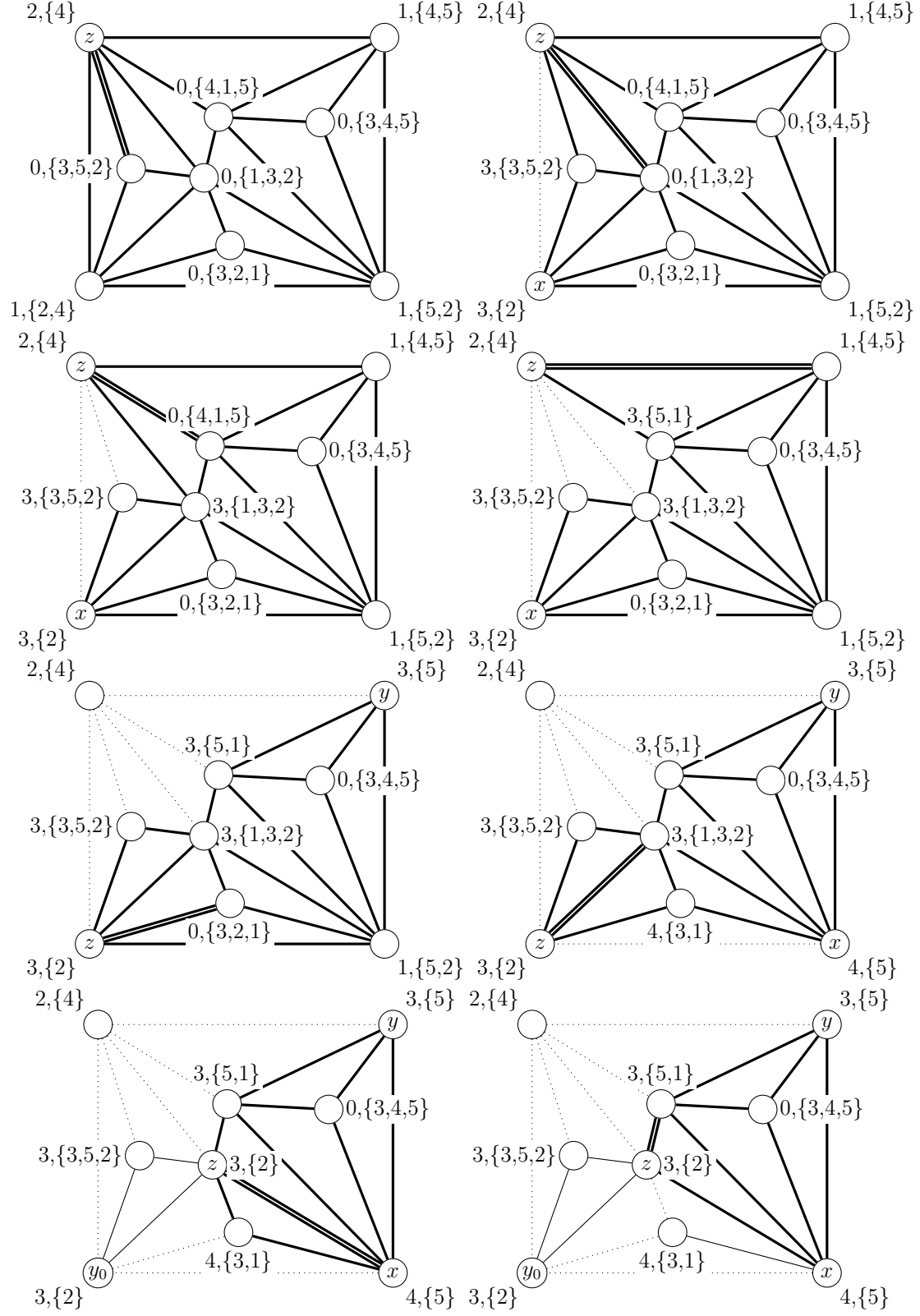


Figure 9: An example of Algorithm 4.2. Each vertex $v \in G$ is labeled with $S(v)$, $L(v)$.

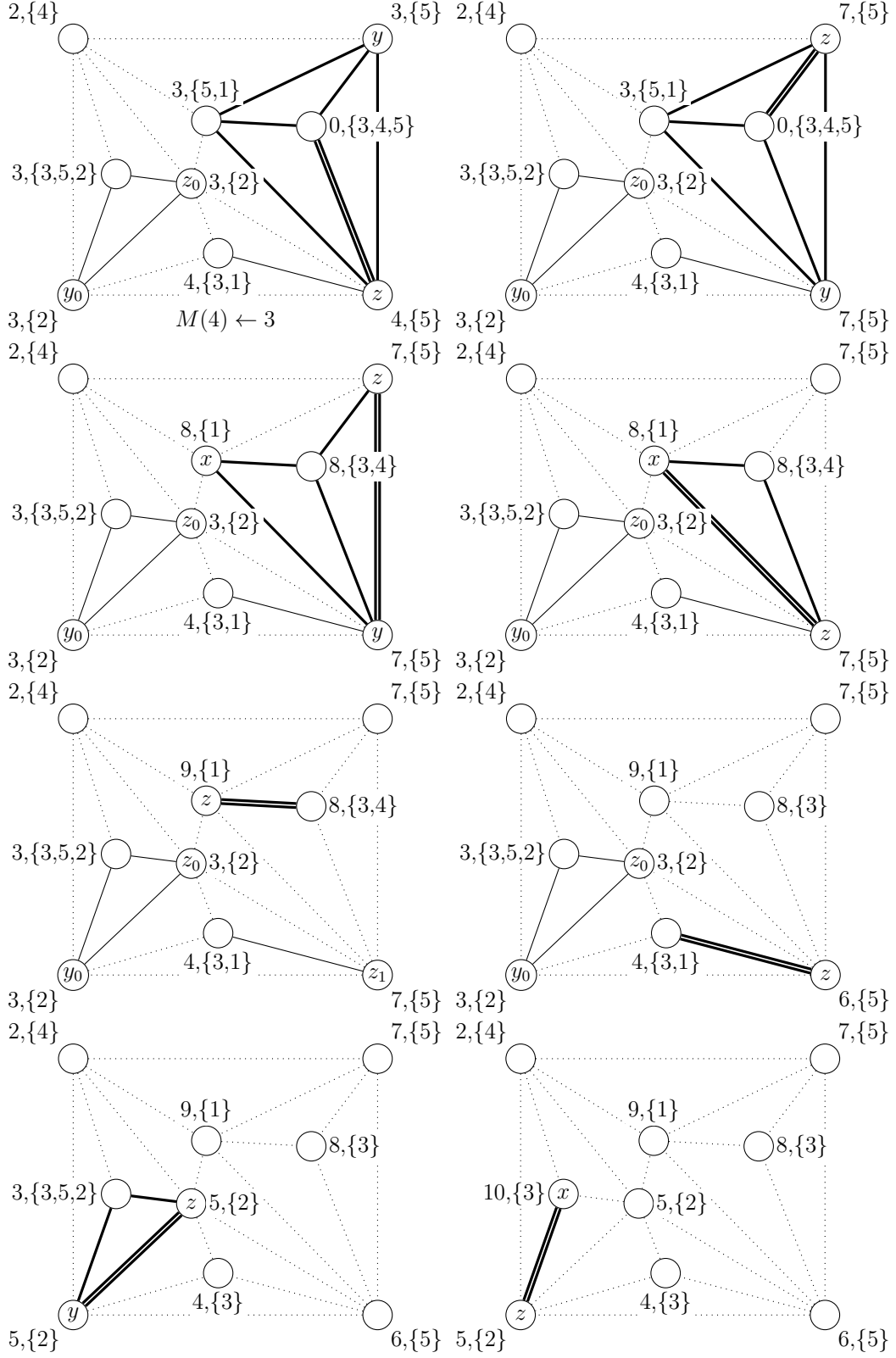


Figure 10: An Algorithm 4.2 example, continued from Figure 9.

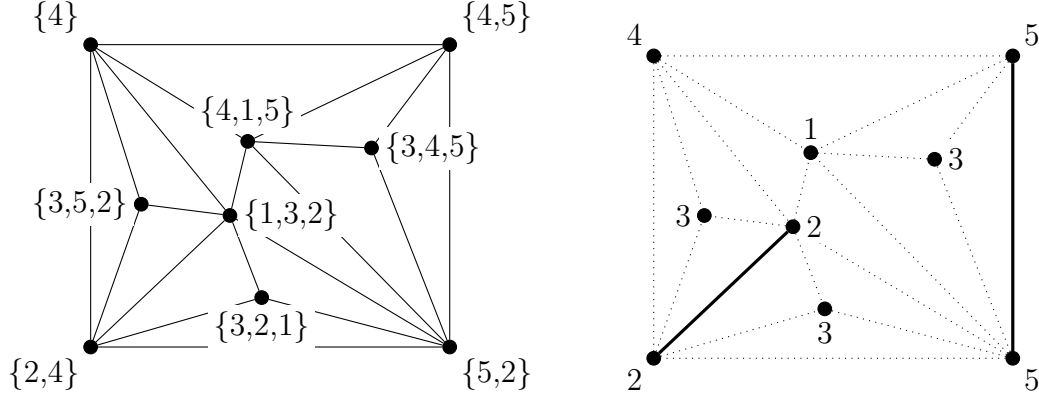


Figure 11: The initial list assignment L (left) and the induced paths of the resulting path L -coloring (right) from the Algorithm 4.2 example shown in Figure 9 and Figure 10.

	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
(Black) BFS	$9.72 \cdot 10^4$	$2.82 \cdot 10^5$	$3.94 \cdot 10^6$	$1.04 \cdot 10^8$	$1.31 \cdot 10^9$
(Green) Algorithm 2.1	$3.35 \cdot 10^4$	$5.34 \cdot 10^5$	$2.10 \cdot 10^7$	$4.15 \cdot 10^8$	$4.65 \cdot 10^9$
(Brown) Algorithm 3.2	$2.24 \cdot 10^5$	$4.02 \cdot 10^6$	$6.78 \cdot 10^7$	$1.99 \cdot 10^9$	$3.57 \cdot 10^{10}$
(Red) Algorithm 3.3	$4.63 \cdot 10^4$	$5.99 \cdot 10^5$	$7.38 \cdot 10^6$	$1.58 \cdot 10^8$	$1.83 \cdot 10^9$
(Blue) Algorithm 4.2	$4.59 \cdot 10^4$	$6.46 \cdot 10^5$	$9.89 \cdot 10^6$	$1.84 \cdot 10^8$	$2.12 \cdot 10^9$

Figure 12: Average time (ns) per triangulated plane graph (sample size 100).

The performance of Algorithm 4.2 was similar to Algorithm 3.3 on the sampled graphs, but Algorithm 4.2 required an augmented adjacency list representation. For situations where such a representation is not immediately available, the time required by Algorithm 4.2 would also include the time required by Algorithm 2.1.

A benchmark was also run to test the asymptotic performance of Algorithm 3.2 and Algorithm 3.3 on the collection of pyramid graphs $\{A_k\}_{k \in \mathbb{N}}$ from Figure 3. Algorithm 3.2 scaled very poorly on pyramids, as expected; see the timings in Figure 13 and plot in Figure 14. Once again, vertex labels were uniformly shuffled. For each $n = 10^i$ we selected the closest integer k to $\sqrt{2n-6}$ so that $n(A_k) \approx n$.

Algorithm 3.3 and Algorithm 4.2 perform recursive calls that operate on subgraphs that are disjoint except for select vertices on their respective outer faces. Therefore it is

	$n \approx 10^3$	$n \approx 10^4$	$n \approx 10^5$	$n \approx 10^6$	$n \approx 10^7$
(Brown) Algorithm 3.2	$2.59 \cdot 10^5$	$8.71 \cdot 10^6$	$4.78 \cdot 10^8$	$7.38 \cdot 10^{10}$	$2.95 \cdot 10^{12}$
(Red) Algorithm 3.3	$1.95 \cdot 10^4$	$2.07 \cdot 10^5$	$3.49 \cdot 10^6$	$1.48 \cdot 10^8$	$2.26 \cdot 10^9$

Figure 13: Average time (ns) per pyramid graph from Figure 3 (sample size 30).

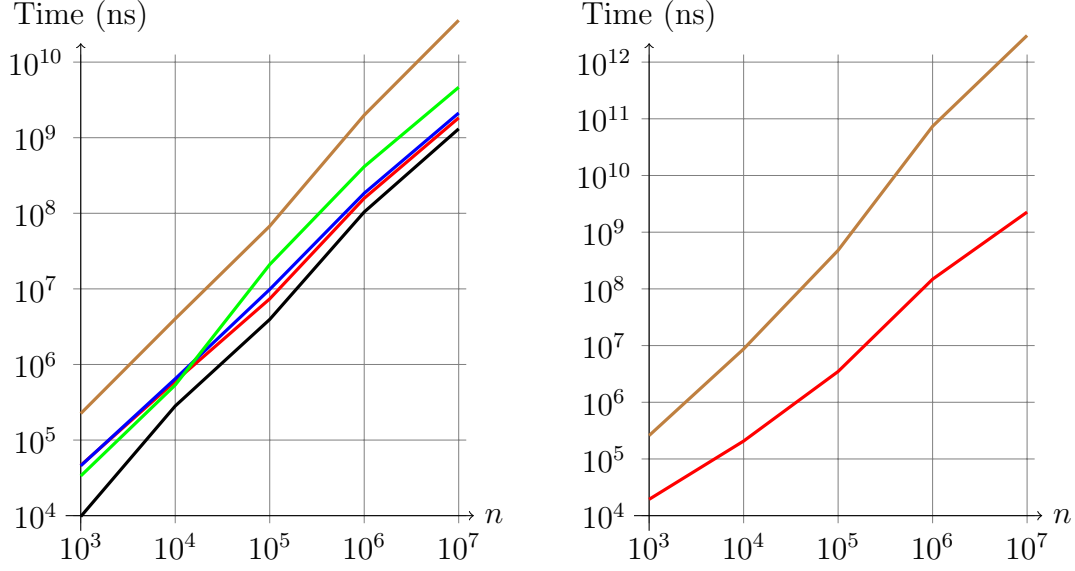


Figure 14: Plots of Figure 12 (left) and Figure 13 (right).

Threads	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
1	$4.63 \cdot 10^4$	$5.99 \cdot 10^5$	$7.38 \cdot 10^6$	$1.58 \cdot 10^8$	$1.83 \cdot 10^9$
2	$4.04 \cdot 10^4$	$4.61 \cdot 10^5$	$4.49 \cdot 10^6$	$9.04 \cdot 10^7$	$1.04 \cdot 10^9$
3	$4.12 \cdot 10^4$	$3.58 \cdot 10^5$	$3.50 \cdot 10^6$	$6.75 \cdot 10^7$	$7.83 \cdot 10^8$
4	$4.32 \cdot 10^4$	$3.14 \cdot 10^5$	$2.98 \cdot 10^6$	$5.59 \cdot 10^7$	$6.53 \cdot 10^8$

Figure 15: Parallel Algorithm 3.3 average time (ns) per graph (sample size 100).

possible to maintain a stack of independent recursive frames and have a pool of threads operate on the frames concurrently. The amount of parallelism possible with this method is highly dependent on the structure of the graph. E.g. when Algorithm 3.3 is run on a pyramid graph from Figure 3 there will never be more than one recursive frame on the stack, and thus no opportunity for parallel execution.

In the case of Algorithm 3.3 such parallelism can easily be organized since each frame writes only to vertices interior to the outer face of its associated subgraph. Therefore the implementation can maintain a shared lock-guarded stack from which all threads push and pop recursive frames. If no frames are available, a thread must idle until either a new frame is pushed to the stack by another thread or until all threads are idle.

To reduce lock contention, each thread should also maintain a small local stack of frames, and only access the shared stack when this local stack is full or empty. For the implementation and hardware used for benchmarks, the most optimal setup proved to be a local stack of 16 frames for each thread. If a local stack filled up, 8 frames were pushed to the shared stack. If a local stack emptied, 8 frames were popped from the shared stack.

Threads	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
1	$4.59 \cdot 10^4$	$6.46 \cdot 10^5$	$9.89 \cdot 10^6$	$1.84 \cdot 10^8$	$2.12 \cdot 10^9$
2	$6.22 \cdot 10^4$	$5.99 \cdot 10^5$	$6.97 \cdot 10^6$	$1.23 \cdot 10^8$	$1.40 \cdot 10^9$
3	$6.09 \cdot 10^4$	$5.02 \cdot 10^5$	$5.65 \cdot 10^6$	$9.32 \cdot 10^7$	$1.06 \cdot 10^9$
4	$6.58 \cdot 10^4$	$4.82 \cdot 10^5$	$5.15 \cdot 10^6$	$7.89 \cdot 10^7$	$8.90 \cdot 10^8$

Figure 16: Parallel Algorithm 4.2 average time (ns) per graph (sample size 100).

Benchmarks in Figure 15 showed benefit to the parallel Algorithm 3.3 when $n \geq 10^4$. The same sample of plane triangulations was used as in Figure 12.

Algorithm 4.2 is trickier to adapt to parallel execution. The first difficulty to consider is that the recursive calls made in Algorithm 4.2 relied on a sequential execution order. If we save a local copy of $S(v), B(v), F(v)$ for each $v \in \{x, y, z\}$ alongside each frame, then the recursive calls in Case 3.2.1 with $u_2 = x$, Case 3.2.2, Case 3.2.3.1, and Case 3.2.3.2 with $u_2 = y$ become order independent. Unfortunately, in Case 3.2.1 with $u_2 \neq x$ and Case 3.2.3.2 with $u_2 \neq y$, the second recursive call will always depend on the reduced color list $L(u_2)$ produced by the first call.

The hurdle of dependent frames can be overcome by maintaining a shared lock-guarded object pool of frames and a stack of frame references. Recursive frames will be added to the pool when they appear in the usual course of the algorithm, but will only be pushed to the shared stack when they are ready to be executed. For each vertex $v \in G$, maintain an optional reference to a frame that will become ready when v has been colored, i.e. has had its color list reduced to length 1. To allow for multiple frames to await the coloring of a single vertex, each frame in the pool will store an “intrusive linked list,” that is, an optional reference to the next frame in line. When a frame is added to the pool and it needs to await a vertex that has another frame waiting on it, the new frame is added to the linked list of the existing frame. When a vertex v is colored, walk the linked list of frames waiting on v and push a reference to each frame onto the shared stack.

Another important detail is that each new face mark $\max(S) + 1$ assigned to S must be unique across all threads, or else we risk corruption and data races against the array M . Uniqueness across threads can be ensured by using a shared atomic counter to track $\max(S)$. Contention can be minimized by having threads reserve marks in chunks, rather than performing an atomic fetch-add operation for each new mark.

In Case 3.2.1, if $u_2 = x$, then $S(x)$ should be assigned a new mark in the recursive frame handling the G_2 subgraph to prevent future writes to $M(S(x))$ from interfering with reads from $M(S(x))$ regarding the vertices on the outer face of G_1 marked with $S(x)$; technically this is optional, as the data race will never affect the outcome of the algorithm. In Figure 9 the mark assigned to the bottom right vertex x of the graph in the step at the bottom right of the page would change from 4 to 7 in the G_2 block. This would then ripple forward, with the next new mark being an 8, and so on.

Lock contention can be reduced the same as in Algorithm 3.3 by having each thread maintain a local stack of frames, but if a frame needs to await the coloring of a vertex, then

it must be added to the shared pool since any thread may color that vertex. Moreover, if a frame colors a vertex that another frame is waiting on, then the lock must be acquired.

Benchmarks in Figure 16 showed some benefit to the parallel version of Algorithm 4.2 when $n \geq 10^4$, with significant performance improvements appearing for very large graphs.

References

- [1] J. Boyer and W. Myrvold, On the cutting edge: simplified $O(n)$ planarity by edge addition, *J. Graph Algorithms Appl.* **8** (2004), 241–273.
- [2] I. Broere and C. M. Mynhardt, Generalized colorings of outerplanar and planar graphs, *Graph theory with applications to algorithms and computer science* (Kalamazoo, Mich., 1984), pp. 151–161, Wiley-Intersci. Publ., Wiley, New York, 1985.
- [3] A. Bross, *Implementing path coloring algorithms on planar graphs*, Masters Project, University of Alaska, 2017, available from http://github.com/permutationlock/path_coloring_bgl and <http://github.com/permutationlock/libavengraph>.
- [4] G. G. Chappell and C. Hartman, Path choosability of planar graphs, *The Electronic Journal of Combinatorics* **25** (2018), P4.33.
- [5] G. Chartrand and H. V. Kronk, The point-arboricity of planar graphs, *J. London. Math. Soc.* **44** (1969), 612–616.
- [6] W. Goddard, Acyclic colorings of planar graphs, *Discrete Math.* **91** (1991), no. 1, 91–94.
- [7] C. M. Hartman, *Extremal Problems in Graph Theory*, Ph.D. Thesis, University of Illinois, 1997.
- [8] J.E. Hopcroft and R.E. Tarjan, Efficient Planarity Testing, *Journal of the ACM* **21** (1974), 549–568
- [9] K. S. Poh, On the linear vertex-arboricity of a planar graph, *J. Graph Theory* **14** (1990), no. 1, 73–75.
- [10] D. Poulalhon and G. Schaeffer, Optimal Coding and Sampling of Triangulations, *Algorithmica* **46** (2006), 505–527
- [11] R. Škrekovski, List improper colourings of planar graphs, *Combin. Probab. Comput.* **8** (1999), no. 3, 293–299.
- [12] D. B. West, *Introduction to Graph Theory*, 2nd ed., Prentice Hall, Upper Saddle River, NJ, 2000.